It’s quite common for a web framework to be used to create an application such as the one shown in listings 5.12 and 5.13. The web framework usually provides additional tools to help separate the different parts of your program. But developing the entire application from scratch has helped you to explore some common patterns for using classes and prototypes in CoffeeScript.

5.8.1. Exercise

The server-side part of the shop application doesn’t use any of the techniques described in this chapter. Write a new version that does use them and compare the two.

5.9. Summary

JavaScript and CoffeeScript are languages with prototype-based objects. JavaScript has always had an uncomfortable relationship with classes. Many people have wanted to add classes to JavaScript; the fourth edition of the ECMAScript specification even proposed adding classes to the language, but that edition was abandoned. With the liberty of making significant syntax changes, CoffeeScript can provide a class syntax that will be familiar to developers from most of the popular class-based object-oriented languages in use today. As you saw in this chapter, the underlying prototype-based model is flexible enough to provide classes.

The choice of whether to use classes at all is up to you. Having learned how they work and by extending them, you’ve learned more about prototypes. In some later chapters you’ll compose objects without classes. Before doing that, though, you’ll return to functions and discover techniques borrowed from functional programming in the next chapter.